ANALOG DEVICES
AHEAD OF WHAT'S POSSIBLE™

Assignment 2

Hardware/Software Co-Design of a NAND Gate

(AND Gate in Programmable Logic, Inversion in Software on the Zynq PS)

Prepared by	Ahmed Yasser
Target Board	Zynq-7000 ZC702 Evaluation Board
EDA Tools	Xilinx Vivado Design Suite / Vitis Unified IDE
Document Type	Assignment Documentation

1. Objective

The goal of this assignment is to implement a NAND gate using a hardware/software co-design approach on the Zynq-7000 platform. An AND gate is implemented as custom logic in the Programmable Logic (PL), while the final inversion needed to turn the AND result into a NAND result is performed in software, via a bare-metal C program running on the Zynq's ARM Processing System (PS).

2. Design Requirements

- A custom RTL AND gate (**and_gate.v**) implemented in the Programmable Logic.
- Two AXI GPIO peripherals bridging the PS and PL: one to read the AND gate's output into software, and one to write the inverted (NAND) result back out to the board.
- A bare-metal C application, run on the Zynq PS, that reads the AND gate output, inverts it, and writes the inverted value to the output GPIO.
- Two switch inputs (a, b) feeding the AND gate, and a single LED output showing the final NAND result.

3. Target Hardware

The design was implemented and verified on a **Zynq-7000 ZC702 Evaluation Board**, which integrates a Zynq-7000 All Programmable SoC (dual-core ARM Cortex-A9 PS tightly coupled with 7-series PL fabric) together with DDR3 memory, Quad-SPI flash, an SD card interface, user push-buttons, an 8-position user DIP switch, and eight user LEDs used to observe the NAND gate's output.

3.1 ZC702 Evaluation Board

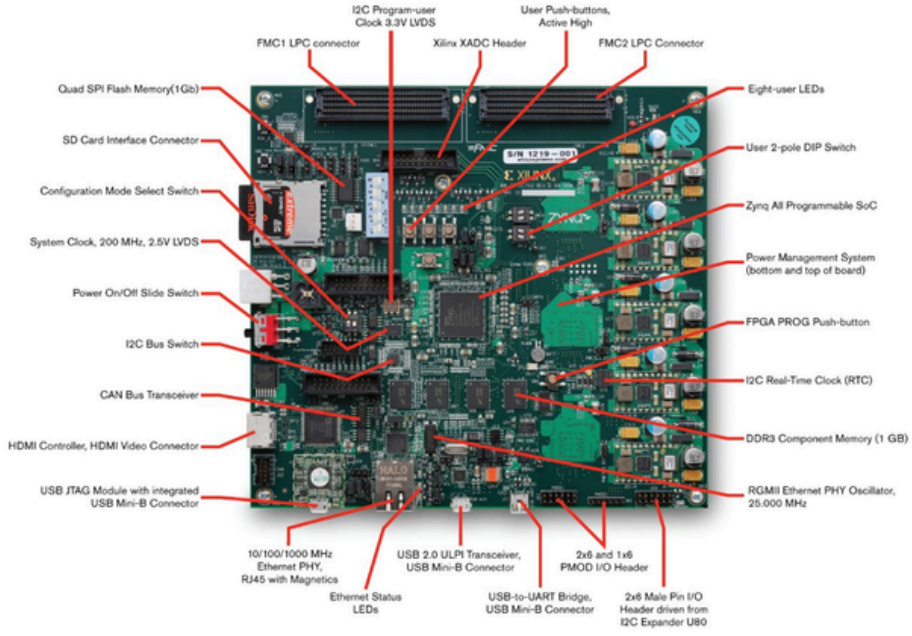


Figure 1: Zynq-7000 ZC702 Evaluation Board and its main on-board resources.

4. Design Architecture

The system combines PL-side custom hardware with PS-side software. The ZYNQ7 Processing System (**processing_system7_0**) connects, through an AXI Interconnect and a Processor System Reset block, to two AXI GPIO peripherals. The two switch inputs (**a_0**, **b_0**) are wired directly into the custom **and_gate_v1_0** RTL IP, whose output **c** is captured by the first AXI GPIO (**axi_gpio_0**) as an input channel readable by software. The C application running on the PS reads this value, inverts it, and writes the result out through the second AXI GPIO (**axi_gpio_1**), whose output channel drives the external port **gpio_io_o_0[0:0]** connected to a user LED - completing the NAND function (AND in hardware, NOT in software).

4.1 Block Design

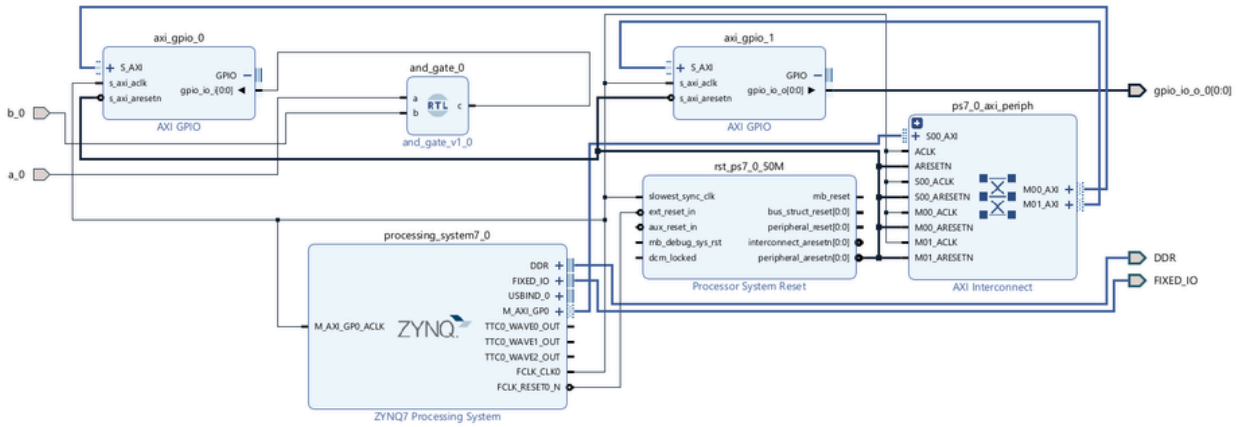


Figure 2: Zynq block design - PS7, AXI Interconnect, Processor System Reset, the two AXI GPIO peripherals, and the custom AND gate IP.

5. Hardware and Software Source Code

This section lists the custom RTL module implementing the AND gate in the Programmable Logic, and the bare-metal C application running on the Zynq Processing System that performs the inversion to realize the NAND function.

5.1 and_gate.v (Programmable Logic)

A simple two-input combinational AND gate, packaged as a custom AXI-independent RTL IP (and_gate_v1_0) and connected directly to the switch inputs in the block design.

```
module and_gate(a, b, c);
    input a;
    input b;
    output c;

    assign c = a & b;
endmodule
```

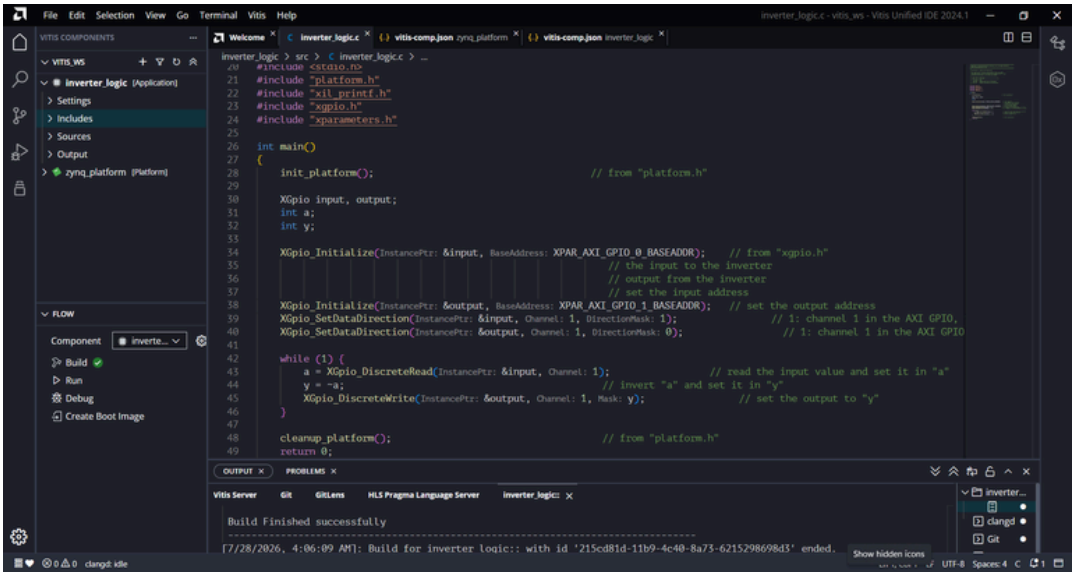
5.2 inverter_logic.c (Zynq Processing System)

A bare-metal C application built in Vitis that continuously reads the AND gate's output through **axi_gpio_0**, inverts it in software, and writes the inverted value out through **axi_gpio_1**, producing the NAND output on the board.

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xgpio.h"
#include "xparameters.h"

int main()
{
init_platform(); // from "platform.h"

XGpio input, output;
int a;
int y;

XGpio_Initialize(&input, XPAR_AXI_GPIO_0_BASEADDR); // the input to the inverter
XGpio_Initialize(&output, XPAR_AXI_GPIO_1_BASEADDR); // output from the inverter
XGpio_SetDataDirection(&input, 1, 1); // channel 1 in the AXI GPIO, set as input
XGpio_SetDataDirection(&output, 1, 0); // channel 1 in the AXI GPIO, set as output

while (1) {
a = XGpio_DiscreteRead(&input, 1); // read the input value and set it in "a"
y = ~a; // invert "a" and set it in "y"
XGpio_DiscreteWrite(&output, 1, y); // set the output to "y"
}

cleanup_platform(); // from "platform.h"
return 0;
}
```

5.3 Vitis Build

The application was built in the Vitis Unified IDE against the exported hardware platform (zynq_platform), targeting the AXI GPIO base addresses generated for this design, and built successfully with no errors.



6. I/O Physical Constraints (XDC)

The board-level physical constraints file was adapted for this design to bind the top-level ports to the correct package pins and I/O standards for the two switch inputs and the LED output used to observe the NAND result. The relevant pin assignments used by this design are summarized below.

Signal	Board Resource	Package Pin	I/O Standard
in0	SW0 (DIP switch) → a	F22	LVCMOS18 (Bank 35)
in1	SW1 (DIP switch) → b	G22	LVCMOS18 (Bank 35)
nand_out	LD0 (User LED) → NAND result	T22	LVCMOS33 (Bank 33)

6.1 I/O Standards by Bank

- Bank 33 (nand_out / LED): LVCMOS33.
- Bank 35 (in0, in1 / DIP switches): LVCMOS18 - 1.8V Vadj default setting.

Note: the a_0/b_0 and gpio_io_o_0 signals shown in the block design (Figure 2) correspond to the top-level ports **in0**, **in1**, and **nand_out** after the block design was wrapped and its external ports renamed/connected in the top-level constraints file.

7. Implementation Results

7.1 Timing Summary

Thedesign timing report confirms that all user-specified timing constraints are met, with zero failing endpoints across setup, hold, and pulse-width checks, across the design's 1600 timing endpoints.

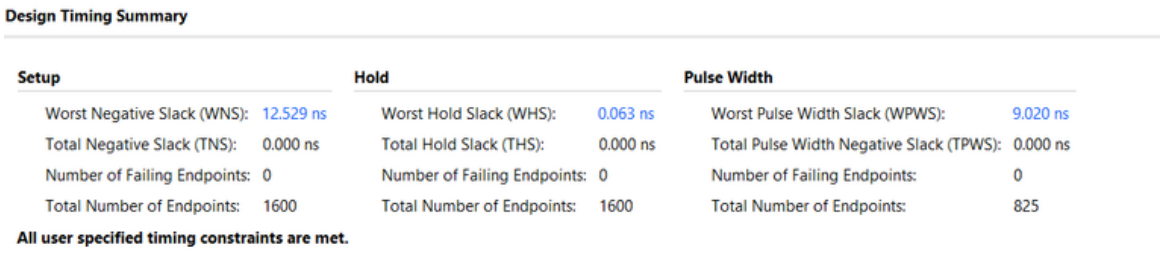


Figure 4: Design timing summary - WNS, WHS and WPWS are all positive with zero failing endpoints.

7.2 Resource Utilization

The overallsystem (Zynq PS7, AXI infrastructure, two AXI GPIO peripherals, and the custom AND gate) uses 572 Slice LUTs and 758 Slice Registers, dominated by the standard AXI GPIO / interconnect infrastructure rather than the trivial AND gate logic itself.

Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOB (200)	Bonded IOPADs (130)	BUFGCTRL (32)
nand_soc_wrapper	572	758	230	510	62	3	130	1
nand_soc_i (nand_soc)	572	758	230	510	62	0	0	1

Figure 5: Post-implementation utilization for the nand_soc wrapper / nand_soc design.

7.3 Device Layout

The implementeddevice view below shows the placement of the AXI GPIO, interconnect, and AND gate logic within the ZC702's Zynq-7000 fabric, alongside the hard Processing System block and its associated clocking resources (BUFGCTRL).

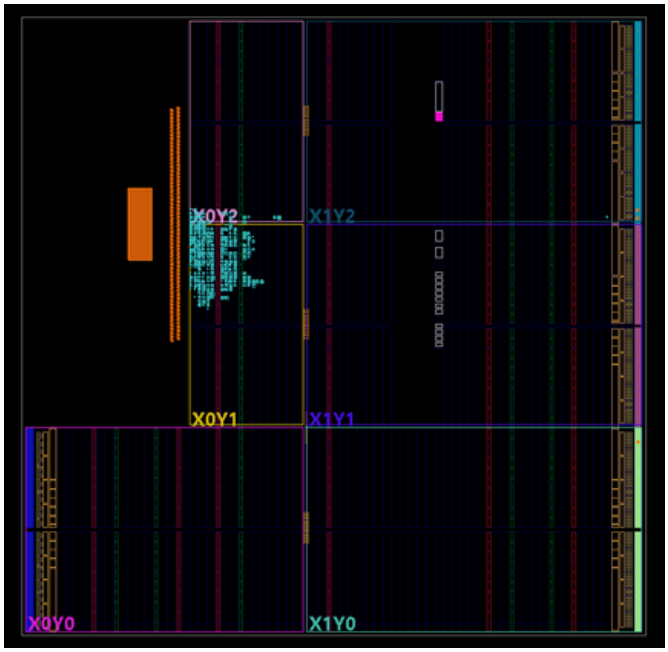


Figure 6: Implemented device layout on the ZC702's Zynq-7000 device.

8. Conclusion

The NAND gate was successfully realized on the Zynq-7000 ZC702 Evaluation Board using a hardware/software co-design approach: the AND operation was implemented as dedicated RTL logic in the Programmable Logic, while the final inversion was performed by a bare-metal C application running on the Zynq Processing System, communicating with the PL through two AXI GPIO peripherals. The Vitis build completed successfully, and implementation results confirm that all timing constraints are met with modest resource utilization, validating the design for on-board verification using the ZC702's DIP switches and user LEDs.